

Q1. Filter Employees by Salary

Given a list of Employee objects, return all employees whose salary is greater than 50,000.

```
class Employee {  
    String name;  
    double salary;  
}
```

Code:

```
import java.util.List;  
import java.util.stream.Collectors;  
  
public class Q1FilterEmployeesBySalary {  
    public static class Employee {  
        private final String name;  
        private final double salary;  
  
        public Employee(String name, double salary) {  
            this.name = name;  
            this.salary = salary;  
        }  
  
        public String getName() {  
            return name;  
        }  
  
        public double getSalary() {  
            return salary;  
        }  
    }  
  
    public static List<Employee> filterBySalary(List<Employee> employees) {  
        return employees.stream()  
            .filter(employee -> employee.getSalary() > 50000)  
            .collect(Collectors.toList());  
    }  
}
```

Q2. Word Frequency Counter

Given a list of words, return a map with the frequency of each word.

Input: ["apple", "banana", "apple"]

Output: {apple=2, banana=1}

Code:

```
import java.util.List;  
import java.util.Map;  
import java.util.function.Function;  
import java.util.stream.Collectors;
```

```

public class Q2WordFrequencyCounter {
    public static Map<String, Long> wordFrequency(List<String> words) {
        return words.stream()
            .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    }
}

```

Q3. Group Employees by Department
Group a list of employees by their department.

```

class Employee {
    String name;
    String department;
}

```

Code:

```

import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

```

```

public class Q3GroupEmployeesByDepartment {
    public static class Employee {
        private final String name;
        private final String department;

        public Employee(String name, String department) {
            this.name = name;
            this.department = department;
        }

        public String getName() {
            return name;
        }

        public String getDepartment() {
            return department;
        }
    }

    public static Map<String, List<Employee>> groupByDepartment(List<Employee> employees) {
        return employees.stream()
            .collect(Collectors.groupingBy(Employee::getDepartment));
    }
}

```

Q4. Employee with Max Salary
Find the employee with the highest salary from the list.

Code:

```
import java.util.Comparator;
import java.util.List;
import java.util.Optional;

public class Q4EmployeeWithMaxSalary {
    public static class Employee {
        private final String name;
        private final double salary;

        public Employee(String name, double salary) {
            this.name = name;
            this.salary = salary;
        }

        public String getName() {
            return name;
        }

        public double getSalary() {
            return salary;
        }
    }

    public static Optional<Employee> findMaxSalary(List<Employee> employees) {
        return employees.stream()
            .max(Comparator.comparingDouble(Employee::getSalary));
    }
}
```

Q5. Join Strings with Comma

Join all strings in a list using a comma as a delimiter.

Input: ["Java", "Python", "C++"]

Output: "Java,Python,C++"

Code:

```
import java.util.List;
import java.util.stream.Collectors;

public class Q5JoinStringsWithComma {
    public static String joinWithComma(List<String> values) {
        return values.stream()
            .collect(Collectors.joining(","));
    }
}
```

Q6. Remove Duplicate Elements

Remove all duplicates from a list of integers.

Code:

```
import java.util.List;
import java.util.stream.Collectors;

public class Q6RemoveDuplicateElements {
    public static List<Integer> removeDuplicates(List<Integer> numbers) {
        return numbers.stream()
            .distinct()
            .collect(Collectors.toList());
    }
}
```

Q7. Partition Employees by Experience

Given a list of employees, partition them into two groups:

- Experience $\geq$ 5 years
- Experience $<$ 5 years

```
class Employee {
    String name;
    int experience;
}
```

Code:

```
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public class Q7PartitionEmployeesByExperience {
    public static class Employee {
        private final String name;
        private final int experience;

        public Employee(String name, int experience) {
            this.name = name;
            this.experience = experience;
        }

        public String getName() {
            return name;
        }

        public int getExperience() {
            return experience;
        }
    }

    public static Map<Boolean, List<Employee>> partitionByExperience(List<Employee> employees) {
        return employees.stream()
            .collect(Collectors.partitioningBy(employee -> employee.getExperience() >= 5));
    }
}
```

Q8. Average Salary by Department

Return a map of department â average salary.

Code:

```
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public class Q8AverageSalaryByDepartment {
    public static class Employee {
        private final String name;
        private final String department;
        private final double salary;

        public Employee(String name, String department, double salary) {
            this.name = name;
            this.department = department;
            this.salary = salary;
        }

        public String getName() {
            return name;
        }

        public String getDepartment() {
            return department;
        }

        public double getSalary() {
            return salary;
        }
    }

    public static Map<String, Double> averageSalaryByDepartment(List<Employee> employees) {
        return employees.stream()
            .collect(Collectors.groupingBy(Employee::getDepartment, Collectors.averagingDouble(getSalary)));
    }
}
```

Q9. Top 3 Salaries

Return the names of the top 3 highest-paid employees.

Code:

```
import java.util.Comparator;
import java.util.List;
import java.util.stream.Collectors;

public class Q9Top3Salaries {
```

```

public static class Employee {
    private final String name;
    private final double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public String getName() {
        return name;
    }

    public double getSalary() {
        return salary;
    }
}

public static List<String> top3HighestPaidNames(List<Employee> employees) {
    return employees.stream()
        .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
        .limit(3)
        .map(Employee::getName)
        .collect(Collectors.toList());
}
}

```

Q10. Sort Employees by Salary (Descending)
Return the list of employees sorted by salary in descending order.

Code:

```

import java.util.Comparator;
import java.util.List;
import java.util.stream.Collectors;

public class Q10SortEmployeesBySalaryDesc {
    public static class Employee {
        private final String name;
        private final double salary;

        public Employee(String name, double salary) {
            this.name = name;
            this.salary = salary;
        }

        public String getName() {
            return name;
        }

        public double getSalary() {
            return salary;
        }
    }
}

```

```

    }

    public static List<Employee> sortBySalaryDesc(List<Employee> employees) {
        return employees.stream()
            .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
            .collect(Collectors.toList());
    }
}

```

Q11. Departments with No Employees

Given a list of departments and a list of employees, return the names of departments with no employees assigned.

Code:

```

import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

public class Q11DepartmentsWithNoEmployees {
    public static class Employee {
        private final String name;
        private final String department;

        public Employee(String name, String department) {
            this.name = name;
            this.department = department;
        }

        public String getName() {
            return name;
        }

        public String getDepartment() {
            return department;
        }
    }

    public static List<String> departmentsWithNoEmployees(List<String> departments, List<Employee> employees) {
        Set<String> assignedDepartments = employees.stream()
            .map(Employee::getDepartment)
            .collect(Collectors.toSet());
        return departments.stream()
            .filter(department -> !assignedDepartments.contains(department))
            .collect(Collectors.toList());
    }
}

```

Q12. Employee Name Initial Map

Return a map where the key is the first character of employee names, and the value is a list of names starting with that character.

Input: ["Alice", "Andrew", "Bob"]
Output: {A=[Alice, Andrew], B=[Bob]}

Code:

```
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.stream.Collectors;

public class Q12EmployeeNameInitialMap {
    public static Map<Character, List<String>> nameInitialMap(List<String> names) {
        return names.stream()
            .filter(Objects::nonNull)
            .filter(name -> !name.isEmpty())
            .collect(Collectors.groupingBy(name -> name.charAt(0)));
    }
}
```